

TN Agriculture RAG System

Implementation Blueprint & Step-by-Step Architecture Guide

This technical manual provides a comprehensive review of the step-by-step procedure implemented to engineer the Retrieval-Augmented Generation (RAG) application for the Tamil Nadu Agriculture & Farmers Welfare Department schemes portal. By shifting from a static web layout to an intelligent, contextual semantic indexing system, the application allows users to query complex government policies naturally.

1. Architectural Workflow

The system follows a decoupling approach, cleanly separating the intensive **Data Ingestion Pipeline** (offline processing) from the high-velocity **User Query & LLM Synthesis Loop** (runtime execution).

System Architecture & Data Flow Diagram

```
[Web Portal: TN Gov Scheme List]
  | (BeautifulSoup Level 1 & 2 Scraping)
  v
[Raw Text Extraction & LangChain Documents]
  | (RecursiveCharacterTextSplitter)
  v
[Text Chunks + Metadata Preserved]
  | (OpenAI text-embedding-3-small)
  v
[FAISS Vector Index Saved Locally (faiss_index/)]
  |
===== RUNTIME INTERFACE =====
  |
[User Query Input (Streamlit UI)] ---> [FAISS Similarity Retrieval]
                                         |
                                         v
[Streamlit App (RAG Chain UI)] <--- [OpenAI GPT-4o-Mini Synth]
                                   | (Automatic Tracing Active)
                                   v
                                [LangSmith Cloud Platform]
```

2. Step-by-Step Implementation Breakdown

Step 1: Multi-Level Web Scraping & Ingestion

Script: `load_data.py`

The main target webpage acts as an index list. The ingestion module establishes a two-tier extraction framework. First, it scrapes the root URL to discover all anchor nodes referencing sub-pages containing specific parameter targets (``scheme_details.php?id=...``). Second, it iterates sequentially through each target, parses structural blocks, and isolates vital text fields such as funding configurations, eligibility data, and application routing targets.

Step 2: Document Custom Packaging & Normalization

Script: Included within ingestion phase

Extracted data fields are mapped safely into standardized LangChain Document structural units. This architecture separates the text data (`page_content`) from programmatic data descriptors (`metadata`). Capturing the unique configuration rules, title names, and direct URLs at this stage ensures that eventual source attribution and audit validation remain deterministic.

Step 3: Document Context Chunking & Windowing

Script: `build_full_knowledge_base.py`

Large text models are vulnerable to severe degradation if sent massive contextual dumps at runtime. To mitigate this context limit constraint, a `RecursiveCharacterTextSplitter` configuration isolates fragments using sequential whitespace and newline checkpoints. A strategic structural window size of 500 characters with a 50-character sliding boundary overlap guarantees that syntactic fragments crossing boundaries preserve their logical meaning.

Step 4: Vector Generation & FAISS Index Compilation

Script: `build_full_knowledge_base.py`

Normalized text fragments are transformed into high-dimensional vector representations using OpenAI's high-efficiency `text-embedding-3-small` model. These mathematical profiles are loaded directly into a local **FAISS (Facebook AI Similarity Search)** vector storage array. The local matrix saves an optimized binary coordination structure (`index.faiss`) alongside a strict key-value lookup system (`index.pkl`).

Step 5: Modular App Configuration & Secret Encapsulation

Script: `.env`

Security credentials, operational endpoints, and structural parameters are extracted completely outside the functional code block into an environmental variable container. Utilizing the `python-dotenv` wrapper ensures parameters remain highly secure and flexible across deployment pipelines.

Step 6: Streamlit UI Frontend Layout & LCEL Execution Runtime

Script: `app.py`

The user interface combines a clean web interface with a high-speed execution loop built via **LangChain Expression Language (LCEL)**. When a user supplies a search query, the app performs a local FAISS semantic similarity fetch to yield the top 3 contextual components. These elements feed directly into a custom `ChatPromptTemplate` sent to OpenAI's `gpt-4o-mini` model for factual, structured synthesis.

Step 7: Production Observability, Metrics & Telemetry Tracking

Integration: Native Environment Binding

By establishing active trace variables within the core runtime container, every step of the RAG sequence ships operational logs seamlessly to the **LangSmith** observability service. Developers can accurately inspect token budgets, prompt alignment errors, execution latencies, and vector search performance in real-time.

3. Project File Directory Matrix

- `.env` - Encapsulates private environment API bindings and configuration flags.
- `load_data.py` - Core web extraction engine that handles HTTP targets and returns Document structures.
- `build_full_knowledge_base.py` - Performs bulk scraping, handles chunk splitting, generates embeddings, and compiles the local FAISS index.
- `app.py` - Orchestrates the visual Streamlit frontend, runtime LCEL pipeline execution, and vector-backed synthesis.
- `faiss_index/` - Permanent local vector index directory (holds `index.faiss` and `index.pkl`).